

BSDA2001P: Introduction to DL and GenAI Project

Smart MCQ Solver Project Report

Name- Mrinal Pandey
Roll no- 23f2000333

1. Abstract

The Smart MCQ Solver Challenge focuses on automatically predicting the correct answer to multiple-choice questions using deep learning and modern natural language processing techniques. Each question consists of a textual prompt and five candidate answer options, with the objective of generating the top three ranked predictions evaluated using the Mean Average Precision at 3 (MAP@3) metric.

This project investigates three different modeling paradigms to address the problem: a custom-built Bidirectional Long Short-Term Memory (BiLSTM) network, a pretrained CrossEncoder transformer model, and a parameter-efficient fine-tuned Qwen2.5-7B-Instruct large language model using Low-Rank Adaptation (LoRA). The BiLSTM serves as the baseline deep learning architecture built entirely from scratch, while the CrossEncoder demonstrates the effectiveness of transfer learning through pretrained contextual representations. The final model leverages instruction-tuned large language models combined with 4-bit NormalFloat (NF4) quantization and LoRA fine-tuning to efficiently adapt a 7-billion-parameter model to the multiple-choice reasoning task.

Among the evaluated models, the fine-tuned Qwen2.5-7B-Instruct model achieved the best overall performance, obtaining a Public Leaderboard MAP@3 score of 0.75519, outperforming both the BiLSTM baseline (0.74023) and the pretrained CrossEncoder (0.44887).

The study demonstrates that combining parameter-efficient fine-tuning with ensemble-based inference strategies can substantially improve multiple-choice question answering performance while maintaining computational efficiency.

2. Introduction

2.1 Problem Statement

Multiple-choice question answering (MCQA) is a fundamental task in natural language understanding that requires a model to interpret a question, reason over multiple candidate answers, and identify the most appropriate option. Unlike traditional text classification problems, MCQA requires contextual comprehension, semantic reasoning, and discrimination between closely related answer choices. The Smart MCQ Solver Challenge provides a dataset containing question prompts accompanied by five candidate answers labelled **A–E**. Rather than predicting a single answer, participants are required to rank the answer options, with evaluation performed using the **Mean Average Precision at 3 (MAP@3)** metric. This evaluation strategy rewards models that correctly identify the ground-truth answer within their top three predictions while also considering the ranking order.

2.2 Project Objectives

The objectives of this project were:

- To investigate the effectiveness of different deep learning paradigms for multiple-choice question answering.

- To compare the performance of a custom-built recurrent neural network, a pretrained transformer model, and a fine-tuned large language model.
- To explore parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) for adapting large language models on limited computational resources.
- To evaluate different inference strategies and improve prediction quality through ensemble-based confidence aggregation.
- To achieve competitive performance on the Kaggle leaderboard while developing a comprehensive understanding of modern NLP model development workflows using PyTorch, Hugging Face Transformers, PEFT, and Weights & Biases.

2.3 Report Structure

The remainder of this report is organized as follows. Section 3 describes the dataset, exploratory analysis, and preprocessing techniques employed during experimentation. Section 4 discusses the tokenization strategies adopted for different model architectures. Section 5 presents the three implemented models, including their architectures, training methodology, and implementation details. Section 6 compares the experimental results using both validation metrics and Kaggle leaderboard performance. Finally, Section 7 summarizes the key findings, discusses challenges encountered during development, and outlines potential directions for future work.

3. Dataset & Preprocessing

3.1 Dataset Description

The Smart MCQ Solver Challenge dataset consists of multiple-choice questions, where each sample contains a textual question prompt, five candidate answer options labelled A–E, and the corresponding ground-truth answer for the training set. The objective is to predict the three most probable answer options for each question, with performance evaluated using the Mean Average Precision at 3 (MAP@3) metric. Each record contains the following fields:

Id, prompt ,A, B, C, D, E, answer

The dataset contains **2,000 labelled training samples** and a separate test set contains 500 samples without labels.

3.2 Exploratory Data Analysis (EDA)

An initial exploratory analysis was performed to better understand the characteristics of the dataset before model development. The analysis focused on:

- Distribution of answer classes (A–E)
- Prompt length distribution
- Missing value analysis
- Class balance
- Text complexity across question prompts

The dataset exhibited a balanced distribution of answer labels, reducing the need for class-balancing techniques. Prompt lengths varied considerably, indicating that models capable of capturing long-range contextual dependencies would likely perform better than traditional bag-of-words approaches. No significant missing values or duplicate records were observed, allowing the dataset to be used directly after basic preprocessing.

3.3 Data Preprocessing

A lightweight preprocessing pipeline was adopted to preserve the semantic information present in the questions while ensuring compatibility with different model architectures. The preprocessing pipeline included:

- Standardization of textual inputs
- Formatting question–option pairs into model-specific input representations
- Label encoding of answer classes for supervised training
- Construction of training and validation splits for model evaluation

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

Different tokenization strategies were adopted for the three models based on their underlying architectures.

The custom BiLSTM model utilized a vocabulary generated from the training corpus to convert textual inputs into integer token sequences suitable for embedding-based learning.

The pretrained CrossEncoder employed the tokenizer associated with its underlying transformer architecture, enabling the joint encoding of question–answer pairs while preserving contextual relationships between tokens.

The best-performing model, Qwen2.5-7B-Instruct, utilized the official Qwen tokenizer provided through the Hugging Face Transformers library. This tokenizer is based on Byte Pair Encoding (BPE), which efficiently represents both common and rare words through subword segmentation while maintaining compatibility with the pretrained language model.

4.2 Justification

Appropriate tokenization plays a crucial role in determining model performance. While vocabulary-based tokenization is sufficient for recurrent neural networks, transformer-based language models rely on pretrained subword vocabularies that capture morphological variations and unseen words more effectively. Using the native tokenizer for each pretrained model ensures consistency between pretraining and downstream fine-tuning, thereby improving contextual understanding and overall prediction accuracy.

4.3 Implementation

Tokenization was performed using the Hugging Face Transformers library with batched processing to improve computational efficiency. Inputs were tokenized using fixed maximum sequence lengths with automatic padding and truncation to ensure uniform tensor dimensions during training and inference. For the Qwen model, left-padding was employed to align with causal language modeling requirements.

4.4 Experimental Comparison

Multiple tokenization approaches were evaluated throughout the project. Vocabulary-based tokenization was suitable for the BiLSTM baseline but lacked the contextual richness required for complex reasoning tasks. Transformer-specific tokenizers consistently produced stronger representations, with the Qwen tokenizer providing the best downstream performance when combined with LoRA fine-tuning and

ensemble inference. Consequently, the Qwen tokenizer was selected as the final tokenization strategy for the highest-performing model.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer Model (Qwen2.5-7B-Instruct with LoRA)

Architecture

The final and best-performing model is based on Qwen2.5-7B-Instruct, a decoder-only transformer language model developed by Qwen Team, *Qwen2.5 Technical Report*, Alibaba Cloud, 2024. Instead of full-parameter fine-tuning, the model was adapted using Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning technique that introduces trainable low-rank matrices into selected transformer layers while keeping the original pretrained weights frozen. To reduce GPU memory requirements, the base model was loaded using 4-bit NormalFloat (NF4) quantization, enabling efficient fine-tuning of a 7-billion-parameter model on limited hardware.

Salient Features

The transformer architecture employs self-attention mechanisms to model long-range contextual relationships between tokens, making it highly effective for complex language understanding and reasoning tasks. The use of LoRA significantly reduces the number of trainable parameters while preserving the knowledge learned during large-scale pretraining.

To improve prediction robustness, three complementary inference strategies were employed:

- Permuted Letter-Logit (PLM) Scoring to reduce positional bias among answer options.
- Chain-of-Thought (CoT) prompting to encourage explicit intermediate reasoning before prediction.
- Verification-based scoring, where the model independently evaluates candidate answers to improve confidence estimation.

The outputs from these strategies were combined using a weighted ensemble to generate the final answer ranking.

Fine-Tuning Strategy

Fine-tuning was performed using the Hugging Face Transformers and PEFT libraries with the AdamW optimizer. LoRA adapters were inserted into the attention projection layers, while the remaining pretrained parameters remained frozen. The model was trained using mixed-precision computation and gradient accumulation to improve computational efficiency.

Quantization

NF4 4-bit quantization was employed through the BitsAndBytes library to substantially reduce GPU memory consumption while maintaining model performance. This enabled efficient fine-tuning and inference without significant degradation in predictive accuracy. The final model achieved the highest Kaggle Public Leaderboard score of **0.75519 MAP@3**, outperforming all other evaluated architectures.

5.2 Model 2: Pretrained Model(CrossEncoder)

Architecture

The CrossEncoder model was implemented using a pretrained transformer encoder that jointly processes the question prompt and a candidate answer option as a single input sequence. Unlike bi-encoder architectures, which encode the question and answer independently, the CrossEncoder computes contextual interactions between both sequences during every forward pass, enabling richer semantic representations. For each question, separate prompt–option pairs were constructed for all five answer choices, and the model predicted a relevance score for each pair. The answer options were subsequently ranked based on these scores.

Salient Features

The CrossEncoder leverages contextual attention across both the question and candidate answer, allowing it to capture fine-grained semantic relationships that are often missed by traditional embedding-based methods. Its primary advantage lies in its strong contextual understanding without requiring handcrafted feature engineering. However, because each answer option must be evaluated independently, inference is computationally more expensive than encoder-based retrieval models.

Training Strategy

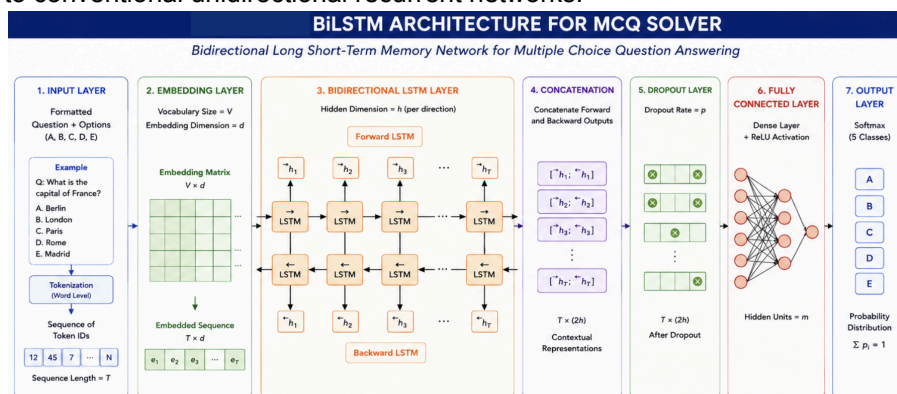
The model was fine-tuned using supervised learning with labelled prompt–answer pairs generated from the training dataset. AdamW optimization, early stopping, and validation-based model selection were employed to reduce overfitting and improve generalization.

Despite benefiting from pretrained transformer representations, the CrossEncoder achieved a comparatively lower Public Leaderboard MAP@3 score of **0.44887**, indicating that straightforward relevance classification was insufficient for the complex reasoning required by the competition.

5.3 Model 3: Custom Deep Learning Model(Bidirectional LSTM)

Architecture

A Bidirectional Long Short-Term Memory (BiLSTM) network was implemented as the baseline recurrent neural network architecture. The model consists of an embedding layer followed by bidirectional LSTM layers, dropout regularization, and a fully connected classification layer with softmax activation. By processing the input sequence in both forward and backward directions, the BiLSTM captures contextual information from preceding as well as succeeding words, improving sequence representation compared to conventional unidirectional recurrent networks.



Salient Features

The BiLSTM architecture was selected due to its ability to model sequential dependencies while remaining computationally lightweight compared to transformer-based language models. Dropout regularization was incorporated to reduce overfitting, and pretrained word embeddings enabled effective learning from the relatively small training dataset. Although recurrent architectures are generally less expressive than large transformer models, the BiLSTM demonstrated strong baseline performance, highlighting the effectiveness of sequence modeling for multiple-choice question answering.

Training Strategy

The model was trained using the Adam optimizer with categorical cross-entropy loss. Early stopping based on validation performance was employed to prevent overfitting, while hyperparameters were tuned through multiple experimental iterations. The final BiLSTM achieved a Public Leaderboard MAP@3 score of **0.74023**.

6. Performance & Comparative Analysis

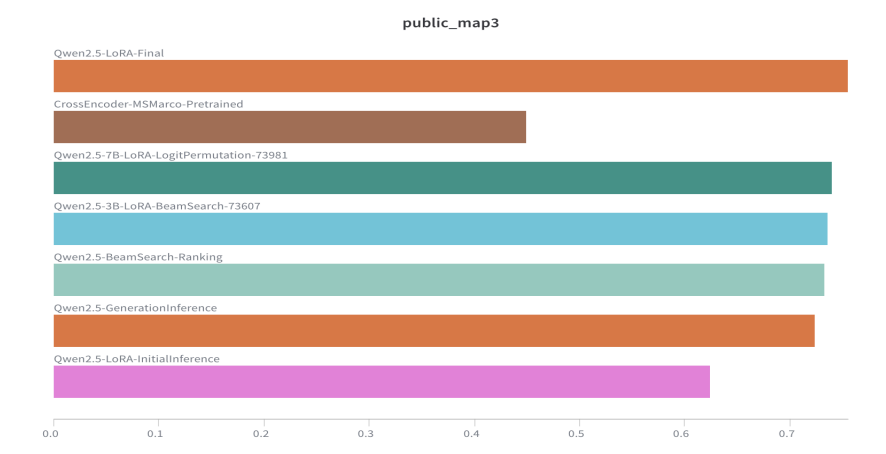
6.1 Evaluation Metrics

The primary evaluation metric for the Smart MCQ Solver Challenge was Mean Average Precision at 3 (MAP@3), which evaluates the quality of the top three predicted answer options for each question. Unlike conventional classification metrics, MAP@3 rewards models that rank the correct answer higher in the prediction list, making it particularly suitable for multiple-choice ranking tasks. During model development, additional metrics such as validation loss and classification accuracy were monitored using Weights & Biases (W&B) to evaluate convergence, compare experimental runs, and assist in hyperparameter selection.

6.2 Training Performance

All three models exhibited stable convergence during training. The BiLSTM demonstrated consistent learning with relatively fast convergence due to its lightweight architecture. The CrossEncoder benefited from pretrained contextual representations but showed limited improvement on the validation set, indicating that pairwise relevance classification alone was insufficient for the competition task.

The fine-tuned Qwen2.5-7B-Instruct model achieved the most stable validation performance. LoRA fine-tuning, together with NF4 quantization, enabled efficient adaptation of the pretrained language model while maintaining low computational overhead. Training and validation metrics were continuously monitored using Weights & Biases, allowing systematic comparison of multiple experimental configurations and facilitating the selection of the final model.



6.3 Comparative Performance

Model	Architecture	Learning Approach	Public MAP@3
BiLSTM	Bidirectional LSTM	Custom Deep Learning	0.74023
CrossEncoder	Transformer Encoder	Pretrained Fine-Tuning	0.44887
Qwen2.5-7B + LoRA	Decoder-only Transformer	Parameter-Efficient Fine-Tuning	0.75519

6.4 Kaggle Performance

The final submission based on the Qwen2.5-7B-Instruct model achieved a Public Leaderboard MAP@3 score of 0.75519, successfully exceeding the competition benchmark established for high-performing submissions.

The final inference pipeline combined Permuted Letter-Logit (PLM) Scoring, Chain-of-Thought (CoT) prompting, and Verification-based confidence estimation through a weighted ensemble, producing more reliable answer rankings than any individual inference strategy alone.

6.5 Error Analysis

Despite the strong overall performance, several error patterns were consistently observed across all models.

Questions requiring extensive external knowledge or domain-specific reasoning remained challenging, particularly when multiple answer options were semantically similar. In several cases, the correct answer appeared within the top three predictions but was assigned a lower confidence score than an incorrect alternative, thereby reducing the MAP@3 score.

The CrossEncoder struggled with questions requiring multi-step reasoning, whereas the BiLSTM occasionally failed to capture long-range contextual dependencies present in lengthy prompts. Although the Qwen-based model substantially reduced these issues through instruction tuning and reasoning-based prompting, errors persisted for highly ambiguous questions where multiple candidate answers were linguistically plausible.

The incorporation of PLM scoring and verification-based confidence estimation mitigated positional bias and improved ranking consistency, contributing significantly to the final performance gain.

7. Conclusion & Future Work

This project explored multiple deep learning approaches for solving the Smart MCQ Solver Challenge, ranging from traditional recurrent neural networks to modern transformer-based large language models. Three different architectures—a Bidirectional LSTM, a pretrained CrossEncoder, and a parameter-efficient fine-tuned Qwen2.5-7B-Instruct model—were implemented, trained, and systematically evaluated using the competition's MAP@3 metric.

Among the evaluated approaches, the fine-tuned Qwen2.5-7B-Instruct model achieved the best overall performance with a Public Leaderboard MAP@3 score of 0.75519. The combination of LoRA-based fine-tuning, NF4 quantization, and an ensemble inference strategy incorporating PLM scoring, Chain-of-Thought reasoning, and verification-based confidence estimation proved highly effective for multiple-choice question answering. While the BiLSTM provided a strong baseline and the CrossEncoder demonstrated the benefits of transfer learning, the large language model exhibited superior reasoning capabilities and produced the most accurate answer rankings.

From this project, valuable experience was gained in modern natural language processing workflows, including parameter-efficient fine-tuning, transformer-based architectures, experiment tracking with Weights & Biases, and large-scale model deployment using the Hugging Face ecosystem. The project also reinforced the importance of prompt engineering and inference-time optimization in improving downstream performance. Future work may focus on incorporating retrieval-augmented generation (RAG) to provide external knowledge during inference, exploring larger instruction-tuned language models, applying automated hyperparameter optimization, and investigating more advanced ensemble techniques to further improve leaderboard performance.

8. References

- [1] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [2] E. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR)*, 2022.
- [3] T. Dettmers *et al.*, "QLoRA: Efficient Finetuning of Quantized LLMs," *NeurIPS*, 2023.
- [4] Qwen Team, *Qwen2.5 Technical Report*, Alibaba Cloud, 2024.

- [5] J. Devlin *et al.*, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.
- [6] Y. Liu *et al.*, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," 2019.
- [7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
- [8] Hugging Face. *Transformers Documentation*. <https://huggingface.co/docs/transformers>
- [9] Hugging Face. *PEFT Documentation*. <https://huggingface.co/docs/peft>
- [10] Kaggle. *Smart MCQ Solver Challenge*. <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>